

Hackathon Challenge

Title

Hyperion — Building an Agentic Assistant for the HYPER-AI IDE

Short Description

HyperAI IDE is the integrated development environment provided by HyperAI to help users build, run, and inspect AI workflows. The goal for this challenge is to develop **Hyperion**, an "agentic" microservice for the HYPER-AI IDE that lets users interact with the IDE using natural language. The agent leverages an LLM to answer general questions about HYPER-AI, control the IDE frontend (create, edit, and delete files), and act on the user's behalf. It is delivered as a Docker image that exposes a URL the IDE talks to.

Context & Motivation

With the rise of GenAI, users increasingly interact with tools through natural language instead of menus and buttons. This challenge brings that shift to the HYPER-AI IDE: making the computing continuum easier to use by replacing manual file editing and navigation with a conversational AI that understands user intent and assists across the full *design* → *deploy* → *monitor* lifecycle.

Related Project

HYPER-AI The HYPER-AI project aims to revolutionise the field of distributed computing by addressing the complexities of integrating Internet of Things (IoT), Edge, and Cloud computing domains. It focuses on the creation of smart virtual computing nodes, optimising data-processing applications across a distributed network to enhance performance and efficiency.

- HyperAI: <https://hyper-ai-project.eu/>
- Live IDE: <https://ide.hyperai.di.uoa.gr/>
- Tutorial: <https://ide-tutorial.hyperai.di.uoa.gr/>

Prerequisites

- **LLM Access:** use the LLM server we provide (per-team access, OpenAI-compatible API) or run your own locally (e.g., Ollama) — see the setup instructions.
- **Language:** any — Python preferred

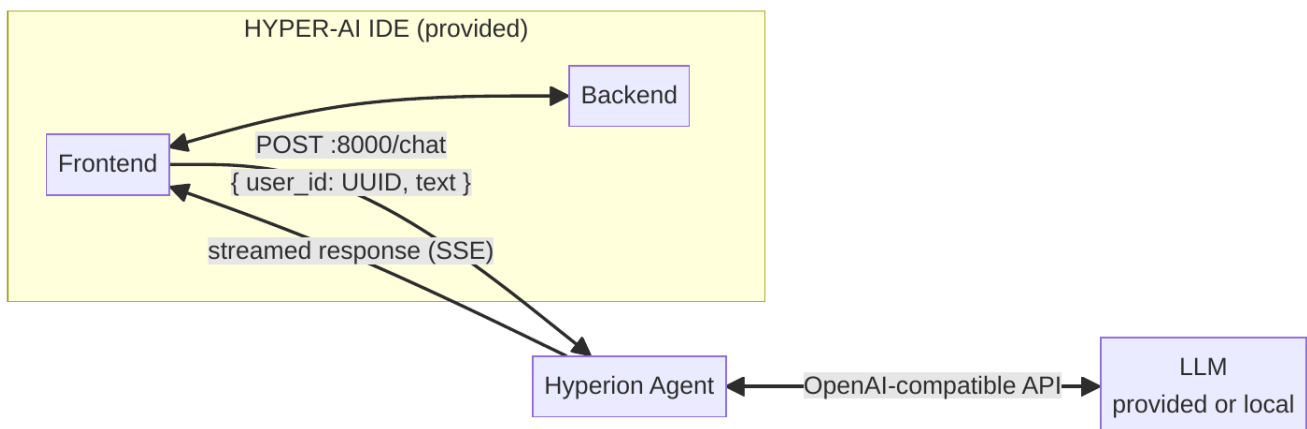
- **Agent framework:** any — LangChain / LangGraph or PydanticAI (Python) preferred;
- **Docker** (the deliverable is a container image)

Expectations

1. A working **microservice agent** — a Docker image that exposes the `/chat` URL — which the IDE talks to. It accurately **answers questions** about HYPER-AI (e.g., "What is HyperAI?") and **translates natural language into IDE actions** (e.g., "Create a deployment YAML for a service using the nginx Docker image" → writes the file and opens it in the editor).
2. **Guardrails:** the system must reject irrelevant queries (e.g., "What is the weather today?").
3. **RAG:** implement a pipeline to ground answers in HYPER-AI documentation.
4. **Memory:** retain context across turns within a session
5. (Optional) **Human-in-the-Loop:** for state-changing actions (e.g., deleting/overwriting a file), the agent *must* implement a confirmation step before execution.

Evaluation: All submissions are scored against the criteria above after the hackathon.

Architecture



The Hyperion agent must expose **port 8000** (the IDE calls `:8000/chat`).

Technical Considerations

Before you start, make sure ports **3001**, **5000** and **8000** are free on your machine.

1. **Open a terminal** and run the `ide-backend` Docker image:

```
docker run --rm -p 3001:3001 -e AUTH_ENABLED=false \  
  --name ide-backend donmichael/ide-backend:latest
```

2. **Open a second terminal** and run the `ide-frontend` Docker image:

```
docker run --rm -p 5000:80 --name ide-gui donmichael/ide-gui:latest
```

3. **Open the IDE** at <http://localhost:5000/>

4. **Create your agent** — start by cloning the `hyperion-starter` repository:

```
git clone https://gitlab.eclipse.org/eclipse-research-labs/hyper-ai-project/hyperion-starter.git
```

Open a third terminal and run it with `uv run main.py` — see the [README in the repository](#) for the full instructions. Your agent is a microservice that listens on **port 8000** with a `/chat` endpoint. On every user prompt the IDE sends `{ "user_id": "...", "text": "..." }`; your response must be **streamed** as Server-Sent Events

- `user_id`: a UUID automatically generated by the IDE — use it to keep per-user session memory
- `text`: the text the user typed in the chat

5. **Package your agent as a Docker image** that runs the service and exposes the URL. If you run `docker compose up --build` from the starter, the image is already built and named `hyperion-starter-hyperion`; tag it and push it to your Docker Hub account (`docker login` first):

```
docker tag hyperion-starter-hyperion <your-dockerhub-user>/hyperion:latest  
docker push <your-dockerhub-user>/hyperion:latest
```

6. **Evaluation**: we run your Docker image (`<your-dockerhub-user>/hyperion:latest`) in a container and perform the evaluation tests against the criteria described above.

Note for Apple Silicon Macs (M1/M2/M3/M4): the `ide-backend` and `ide-gui` images are built for `linux/amd64`. If Docker refuses to start them, add `--platform linux/amd64` to the commands in steps 1 and 2, for example:

```
docker run --rm --platform linux/amd64 -p 5000:80 \  
  --name ide-gui donmichael/ide-gui:latest
```

Example Session

